

A demo of 3D transformers for surgical video guidance

Marios S. Pattichis

Department of Electrical and Computer Engineering

The University of New Mexico, USA

Goal: Guide surgeon to gallbladder

Training Dataset:

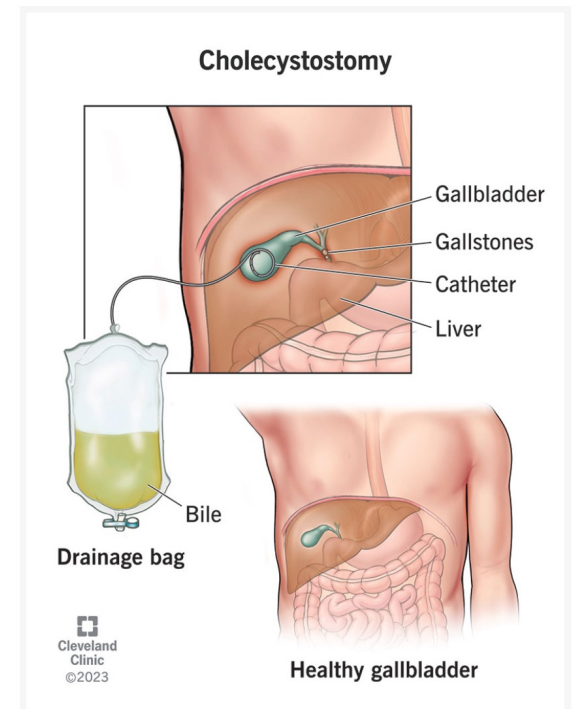
1. Create a simulated video of moving away from the bladder.
2. Set ground truth to opposite direction that moves us closer.

Testing Dataset:

1. Full video.
2. Last frame duplicated.
3. First frame duplicated.
4. Random shuffling of frames.

Architecture:

1. Construct a 3D transformer architecture to process the videos.
2. Add classification token for last output to select among directions.



A cholecystostomy inserts a catheter into your gallbladder to drain excess bile.

Image taken from <https://my.clevelandclinic.org/health/procedures/cholecystostomy>

Simulated video example

answer: move camera down-left ✓
frame 0

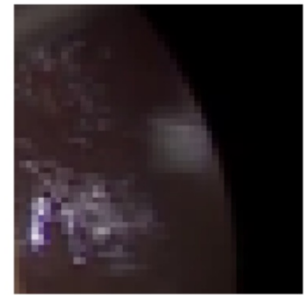
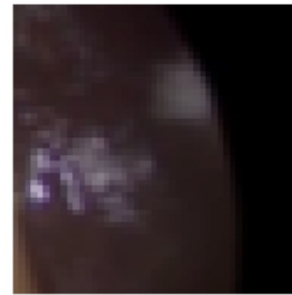
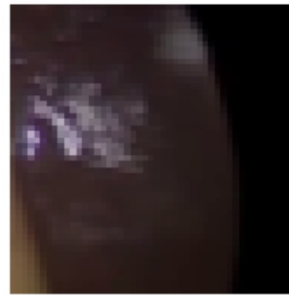
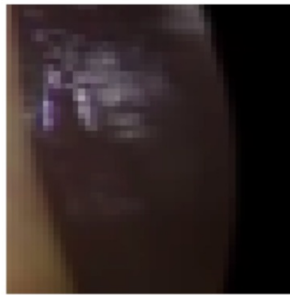
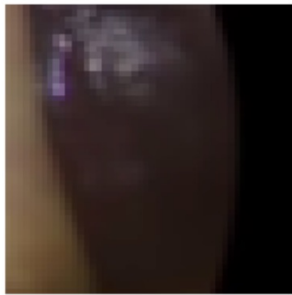
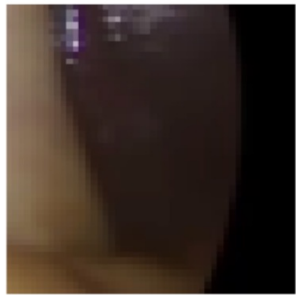
frame 3

frame 6

frame 9

frame 12

frame 15



Input: surgical videos with higher resolution **cropped** to 48x48 frames.

First-frame: Gallbladder located in the lower-left.

Remaining frames: Moving to the upper-right over the full frame. Crop to 48x48.

Ground truth: Correct upper-right movement by recommending down-left.

Limitation: Only 8 directions considered here.

Input video decomposed into segments

```
class TubeletEmbed(nn.Module):
    def __init__(self, in_ch, dim, tubelet):
        super().__init__()
        self.proj = nn.Conv3d(in_ch, dim, kernel_size=tubelet, stride=tubelet)
    def forward(self, x):
        # (B, C, T, H, W)
        x = self.proj(x)
        # (B, dim, T', H', W')
        return x.flatten(2).transpose(1, 2)
        # (B, N = T'H'W', dim)
```

1. Conv3D uses a stride that is the same as the tubelet (=video segment) for non-overlapping segments.
2. At the end, flattening turns them into vectors.

Attention implementation

```
class Attention(nn.Module):
    def __init__(self, dim, heads):
        super().__init__()
        self.heads = heads
        self.qkv = nn.Linear(dim, 3 * dim)
        self.proj = nn.Linear(dim, dim)
    def forward(self, x):
        B, N, C = x.shape
        q, k, v = self.qkv(x).reshape(B, N, 3, self.heads, C // self.heads).permute(2, 0, 3, 1, 4)
        x = F.scaled_dot_product_attention(q, k, v) # fused kernel on GPU
        return self.proj(x.transpose(1, 2).reshape(B, N, C))
```

Dimensions:

B = batch size, N=number of tokens, C=3 for the R, G, B channels.

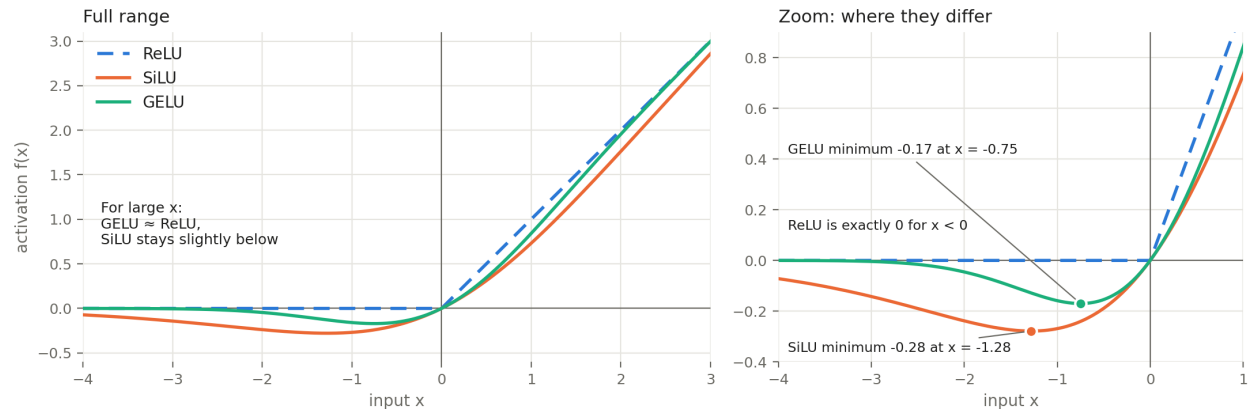
Single video segment = 2 frames of 8x8 blocks each.

F.scaled_dot_product_attention(.) from PyTorch.

Transformer Block

```
class Block(nn.Module):
    def __init__(self, dim, heads, mlp_ratio=4):
        super().__init__()
        self.norm1, self.attn = nn.LayerNorm(dim), Attention(dim, heads)
        self.norm2 = nn.LayerNorm(dim)
        self.mlp = nn.Sequential(nn.Linear(dim, mlp_ratio * dim), nn.GELU(), nn.Linear(mlp_ratio * dim, dim))
    def forward(self, x):
        x = x + self.attn(self.norm1(x))
        return x + self.mlp(self.norm2(x))
```

ReLU vs. SiLU vs. GELU



$\text{ReLU}(x) = \max(0, x)$ $\text{SiLU}(x) = x \cdot \sigma(x)$, $\sigma = \text{sigmoid}$ $\text{GELU}(x) = x \cdot \Phi(x)$, $\Phi = \text{standard normal CDF}$

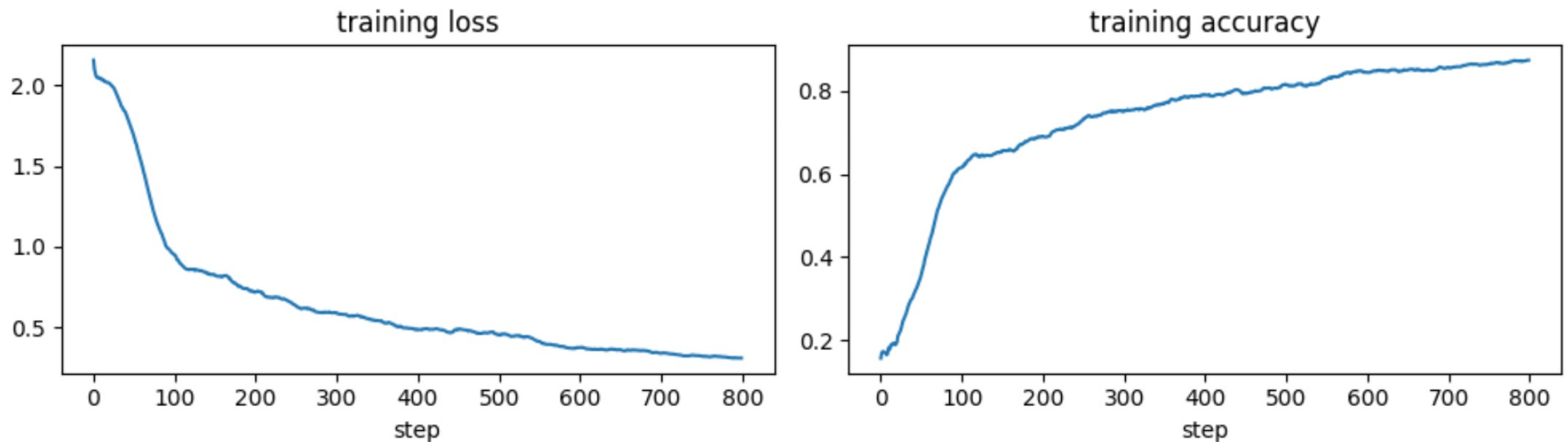
Main Transformer Architecture

```
class ViT3D(nn.Module):
    def __init__(self, input_size=(16, 48, 48), in_ch=3, tubelet=(2, 8, 8),
                  dim=128, depth=4, heads=4, num_classes=8):
        super().__init__()
        self.embed = TubeletEmbed(in_ch, dim, tubelet)
        n_tokens = math.prod(s // p for s, p in zip(input_size, tubelet))
        self.cls = nn.Parameter(torch.zeros(1, 1, dim))
        self.pos = nn.Parameter(torch.randn(1, n_tokens + 1, dim) * 0.02)
        self.blocks = nn.Sequential(*[Block(dim, heads) for _ in range(depth)])
        self.norm = nn.LayerNorm(dim)
        self.head = nn.Linear(dim, num_classes)
    def forward(self, x):
        x = self.embed(x)
        x = torch.cat([self.cls.expand(len(x), -1, -1), x], dim=1) + self.pos
        x = self.norm(self.blocks(x))
        return self.head(x[:, 0])          # classify from the [CLS] token
```

Notes:

1. self.embed(x) implements the video segments.
2. self.pos implements random vectors for position encoding.
3. Self.head(x[:, 0]) trains the classification token to accumulate the results.

The approach works!



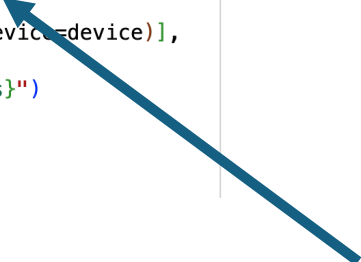
test clips	correct	within 45 deg
full video	80.5%	100.0%
last frame only	57.1%	91.5%
first frame only	21.6%	48.6%
shuffled frames	41.6%	78.1%

Testing the transformer

```
@torch.no_grad()
def evaluate(transform, n=2000):
    model.eval()
    torch.manual_seed(1)
    exact = near = 0
    # the same test clips for every test
    for _ in range(n // 250):
        x, y = make_batch(250, "test")
        with torch.autocast(device_type=device, dtype=torch.float16, enabled=use_amp):
            pred = model(transform(x)).argmax(1)
            off = (pred - y) % 8
            # 0 = correct, 1 or 7 = off by 45 degrees
            exact += (off == 0).sum().item(); near += ((off <= 1) | (off == 7)).sum().item()
    return exact / n, near / n

tests = {
    "full video": lambda x: x,
    "last frame only": lambda x: x[:, :, -1:].expand_as(x),
    "first frame only": lambda x: x[:, :, :1].expand_as(x),
    "shuffled frames": lambda x: x[:, :, torch.randperm(T, device=device)],
}

print(f"{'test clips':20s}{{'correct':>10s}{{'within 45 deg':>16s}}")
for name, fn in tests.items():
    acc, near = evaluate(fn)
    print(f"{'name':20s}{acc:10.1%}{near:16.1%}")
```



Tests resamples the videos and expands to full video length.

Results: Arrow should point to location in first frame

Last frame with the guidance arrow (green = correct, red = wrong)

